



UNIVERSITY OF GENEVA

Faculty of Science

Binary Diffusion Probabilistic Models

Computational Imaging
14x062



Michel Jean Joseph Donnet

2026-05-31



Contents

1	Introduction on Diffusion Models	3
2	Methodology	4
2.1	Forward Diffusion Process	4
2.2	Reverse Diffusion Process	6
2.2.1	Intuitive interpretation	7
3	Binary Diffusion Models	8
3.1	Training the model	8
3.1.1	Loss functions	9
3.1.2	First transformation	9
3.2	Sampling from the model	10
4	Binary Diffusion Models for Tabular Data Generation	11
4.1	Link with image generation	11
4.2	First transformation for tabular data	11
4.3	Architecture of the model	11
4.3.1	Evaluation of the model	12
4.3.2	Fixed thresholding step	12
5	Results of Tabular Data Generation	13
5.1	Removing the fixed thresholding step	13
5.2	Correctly applying the guidance during the sampling process	14
5.3	Fixing the noise addition step in the sampling algorithm	14
5.3.1	Increasing the noise in the sampling process	16
6	Results of Image Generation	18
6.1	Evaluation of the model	18
6.1.1	How to interpret the results of the evaluation	18
6.2	Fixing the thresholding step in the sampling algorithm	18
6.3	Fixing the noise addition step in the sampling algorithm	19
6.4	Increasing the noise in the sampling process	20
6.5	Generated images	20
7	Conclusion	21
7.1	Acknowledgements	21
7.2	Usage of AI	21
	Bibliography	22



1 Introduction on Diffusion Models

Generative models have revolutionized the field of artificial intelligence, enabling machines to create content that is indistinguishable from human-generated data or to improve the quality of data through various transformations. Among these models, diffusion models have emerged as a powerful class of generative models that have shown remarkable performance in tasks such as data generation, image super-resolution, and data denoising.

Diffusion models are based on the concept of a diffusion process, which is a stochastic process that describes how data points evolve over time. The core idea behind diffusion models is to model the data generation process as a gradual transformation from noise to structured data. This is achieved by defining a forward diffusion process that adds noise to the data and a reverse diffusion process that learns to remove the noise and recover the original data.

Among the various types of diffusion models, Denoising Diffusion Probabilistic Models (DDPMs) have gained significant attention due to their ability to generate high-quality samples. DDPMs work by iteratively adding noise to the data and then learning to reverse this process, effectively denoising the data step by step.

But are these models very suitable for all types of data, especially for quantized data? Quantized data, which is often used in scenarios like in computational imaging, presents unique challenges for generative modeling. The discrete nature of quantized data can make it difficult for diffusion models to learn the underlying data distribution effectively.

In answer to this problem, researchers have proposed various approaches to adapt diffusion models for quantized data, such as Binary Diffusion Models (BDMs) and Quantized Diffusion Models (QDMs). These models are designed to handle the discrete nature of quantized data while still leveraging the powerful generative capabilities of diffusion models.

2 Methodology

There are several types of generative models, including:

- Generative Adversarial Networks (GANs)
- Variational Autoencoders (VAEs)
- Normalizing Flows
- Diffusion Models

Each of these models has its own strengths and weaknesses, and the choice of model often depends on the specific application and the type of data being modeled.

Diffusion models, in particular, have shown great promise in generating high-quality samples and have been successfully applied to a wide range of tasks. However, their performance can be limited when dealing with quantized data, which is often encountered in scenarios like computational imaging. To address this issue, researchers have developed specialized diffusion models that are designed to handle quantized data effectively.

Before going into the details of these specialized diffusion models, let's first understand the foundational principles of diffusion models and how they work.

Diffusion models are Markov chains that consist of two main processes: the forward diffusion process and the reverse diffusion process. The forward diffusion process gradually adds noise to the data, while the reverse diffusion process learns to remove the noise and recover the original data. The training of diffusion models involves optimizing a variational bound on the data likelihood, which allows the model to learn the underlying data distribution effectively.

2.1 Forward Diffusion Process

Suppose we have an original image X_0 that follows a data distribution q .

At each time step t , noise is added to the data according to a predefined schedule, which is typically controlled by a parameter β_t that determines the amount of noise added at each step, as shown in Equation 1 where $\varepsilon_t \sim \mathcal{N}(0, I)$ is a standard Gaussian noise term and X_t is the noisy version of the data at time step t .

$$X_t = \sqrt{1 - \beta_t} X_{t-1} + \sqrt{\beta_t} \varepsilon_t \quad (1)$$

Indeed, if $\beta_t = 1$ in Equation 1, then X_t would be pure noise whereas if $\beta_t = 0$, then X_t would be the original data X_{t-1} .

This process can be visualized as a gradual transformation of the original data into noise, where the data becomes increasingly corrupted as more noise is added at each time step, as illustrated in the Figure 1.

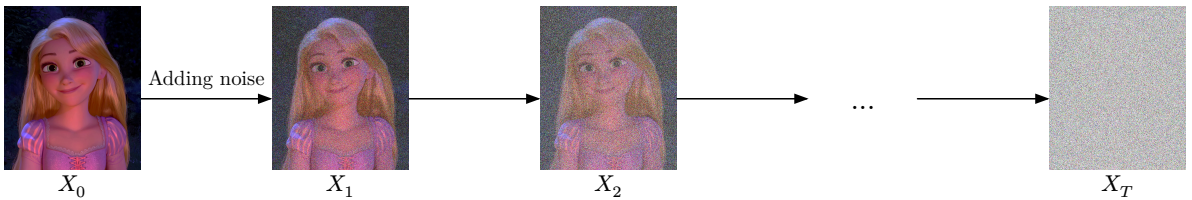


Figure 1: Forward diffusion process



We can also express X_t in term of the data distribution q as described in Equation 2, where $\mathcal{N}(X; \mu, \sigma^2)$ denotes a Gaussian distribution with mean μ and variance σ^2 applied to X .

$$q(X_t | X_{t-1}) = \mathcal{N}(X_t; \sqrt{1 - \beta_t} X_{t-1}, \beta_t I) \quad (2)$$

By iterating this process over T time steps, we can obtain a sequence of noisy data points $(X_1, X_2, \dots, X_T)$, where X_T is the noisiest version of the data which can be considered as pure noise.

This can be further simplified to a closed-form expression that directly relates X_T to the original data X_0 , as shown in Equation 3.

$$q(X_T | X_0) = q(X_1 | X_0) \times q(X_2 | X_1) \times \dots \times q(X_T | X_{T-1}) = \prod_{t=1}^T q(X_t | X_{t-1}) \quad (3)$$

We note $\alpha_t = 1 - \beta_t$, so the forward process can be written as:

$$\begin{aligned} X_t &= \sqrt{\alpha_t} X_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t \\ &= \sqrt{\alpha_t} (\sqrt{\alpha_{t-1}} X_{t-2} + \sqrt{1 - \alpha_{t-1}} \varepsilon_{t-1}) + \sqrt{1 - \alpha_t} \varepsilon_t \\ &= \sqrt{\alpha_t} \sqrt{\alpha_{t-1}} X_{t-2} + \sqrt{\alpha_t} \sqrt{1 - \alpha_{t-1}} \varepsilon_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t \\ &= \sqrt{\alpha_t \alpha_{t-1}} X_{t-2} + \sqrt{\alpha_t (1 - \alpha_{t-1})} \varepsilon_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t \\ &= \sqrt{\alpha_t \alpha_{t-1}} (\sqrt{\alpha_{t-2}} X_{t-3} + \sqrt{1 - \alpha_{t-2}} \varepsilon_{t-2}) + \sqrt{\alpha_t (1 - \alpha_{t-1})} \varepsilon_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t \\ &= \sqrt{\alpha_t \alpha_{t-1} \alpha_{t-2}} X_{t-3} + \sqrt{\alpha_t \alpha_{t-1} (1 - \alpha_{t-2})} \varepsilon_{t-2} + \sqrt{\alpha_t (1 - \alpha_{t-1})} \varepsilon_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t \\ &= \dots \\ &= \sqrt{\prod_{i=1}^t \alpha_i} X_0 + \sum_{k=1}^t \sqrt{(1 - \alpha_k) \prod_{j=k+1}^t \alpha_j} \varepsilon_k \\ &= \sqrt{\bar{\alpha}_t} X_0 + \sum_{k=1}^t \sqrt{(1 - \alpha_k) \prod_{j=k+1}^t \alpha_j} \varepsilon_k \quad \text{with } \bar{\alpha}_t = \prod_{i=1}^t \alpha_i \end{aligned} \quad (4)$$

We note $N_t = \sum_{k=1}^t \sqrt{(1 - \alpha_k) \prod_{j=k+1}^t \alpha_j} \varepsilon_k$. We want to express N_t as a single noise term since a sum of independent Gaussian noise terms is also a Gaussian noise term, which will allow us to obtain a closed-form expression for X_t in terms of X_0 and a single noise term, as shown in Equation 6.

The variance of N_t can be calculated as in Equation 5, where we use the fact that $\varepsilon_1, \varepsilon_2, \dots, \varepsilon_T \sim \mathcal{N}(0, I)$ are independent standard Gaussian noise terms.

$$\begin{aligned}
 \text{Var}(N_t) &= \text{Var}\left(\sum_{k=1}^t \sqrt{(1 - \alpha_k) \prod_{j=k+1}^t \alpha_j} \varepsilon_k\right) \\
 &= \sum_{k=1}^t (1 - \alpha_k) \prod_{j=k+1}^t \alpha_j \text{Var}(\mathcal{N}(0, I)) \\
 &= \sum_{k=1}^t (1 - \alpha_k) \prod_{j=k+1}^t \alpha_j \\
 &= (1 - \alpha_t) + \alpha_t(1 - \alpha_{t-1}) + \alpha_t \alpha_{t-1}(1 - \alpha_{t-2}) + \dots + \prod_{i=2}^t \alpha_i (1 - \alpha_1) \\
 &= 1 - \alpha_t + \alpha_t - \alpha_t \alpha_{t-1} + \alpha_t \alpha_{t-1} - \alpha_t \alpha_{t-1} \alpha_{t-2} + \dots + \prod_{i=2}^t \alpha_i - \prod_{i=1}^t \alpha_i \\
 &= 1 - \prod_{i=1}^t \alpha_i \\
 &= 1 - \bar{\alpha}_t
 \end{aligned} \tag{5}$$

We obtain the final expression for X_t in Equation 6 with $\varepsilon \sim \mathcal{N}(0, I)$ being a single Gaussian noise term with variance $1 - \bar{\alpha}_t$.

$$X_t = \sqrt{\bar{\alpha}_t} X_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \tag{6}$$

This closed-form expression is particularly useful because it allows us to sample X_t directly from X_0 without having to iterate through all the intermediate steps, which can be computationally expensive when T is large.

2.2 Reverse Diffusion Process

The reverse diffusion process consists of going from the noisy data back to the original data by learning to reverse the noise addition process, as illustrated in Figure 2.

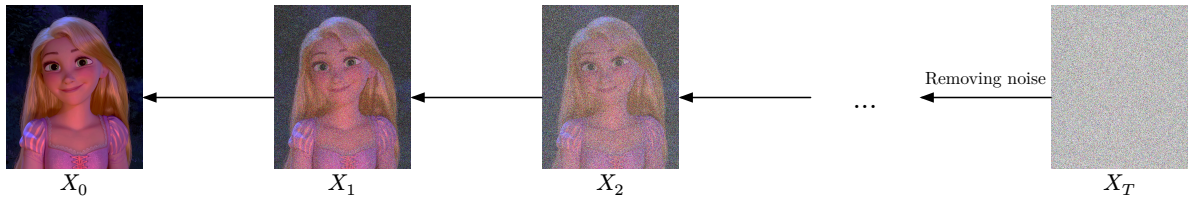


Figure 2: Illustration of the reverse diffusion process

As we cannot estimate $q(X_{t-1} | X_t)$ directly, we train a model $p_\theta(X_{t-1} | X_t)$ to approximate it, where θ denotes the parameters of the model. This conditional distribution can be represented as a Gaussian distribution with mean $\mu_\theta(X_t, t)$ and covariance $\Sigma_\theta(X_t, t)$, as shown in Equation 7.

$$p_\theta(X_{t-1} | X_t) = \mathcal{N}(X_{t-1}; \mu_\theta(X_t, t), \Sigma_\theta(X_t, t)) \approx q(X_{t-1} | X_t) \tag{7}$$

The $\mu_\theta(X_t, t)$ aims to predict the expected value of X_{t-1} given X_t , while $\Sigma_\theta(X_t, t)$ captures the uncertainty in this prediction.

As detailed in the very nice blog post by Lilian Weng on diffusion models [1], the mean $\mu_\theta(X_t, t)$ can be expressed in terms of the noise prediction $\varepsilon_\theta(X_t, t)$ as shown in Equation 8,

where α_t and $\bar{\alpha}_t$ are defined as in the forward diffusion process. In fact, the model aims to predict the noise $\varepsilon_\theta(X_t, t)$ that was added to the data at time step t in the forward diffusion process, and then uses this prediction to compute the mean $\mu_\theta(X_t, t)$ for the reverse diffusion process.

$$\mu_\theta(X_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(X_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(X_t, t) \right) \quad (8)$$

A common choice for the covariance $\Sigma_\theta(X_t, t)$ is to set it to a fixed value, such as $\Sigma_\theta(X_t, t) = \beta_t I$, which corresponds to the variance of the noise added in the forward diffusion process at time step t . Indeed, the covariance will indicate how much noise should be removed at each step of the reverse diffusion process, that's why it is often set to a value that matches the noise level in the forward diffusion process to ensure that the model learns to effectively reverse the noise addition process.

2.2.1 Intuitive interpretation

We can imagine that the data are represented by a mountain with X_0 at the peak. The forward diffusion process can be seen as a hiker starting at the peak and gradually descending the mountain, adding noise to the data at each step. The reverse diffusion process can be thought of as a hiker starting at the base of the mountain (the noisiest data) and trying to climb back up to the peak (the original data) by following the path of least resistance, which is determined by the learned model p_θ .

The $\mu_\theta(X_t, t)$ indicates the direction in which the hiker should move to climb back up the mountain, while $\Sigma_\theta(X_t, t)$ represents the uncertainty in this direction, which indicates how many different paths the hiker can take to reach the peak.

For instance, in the case of image generation, μ_θ indicates that the image should be a bear, while Σ_θ indicates if it should be a black bear, a brown bear, or a polar bear.

So the model aims to minimize the difference between the predicted distribution $p_\theta(X_{t-1} | X_t)$ and the true distribution $q(X_{t-1})$ to learn how to effectively reverse the noise addition process and recover the original data from the noisy data.

The complete process can be represented as in Figure 3, where the model learns to predict the original data $\hat{X}_{0_t}$ and the predicted noise $\hat{z}_t$ from the noisy data X_t at each time step t .

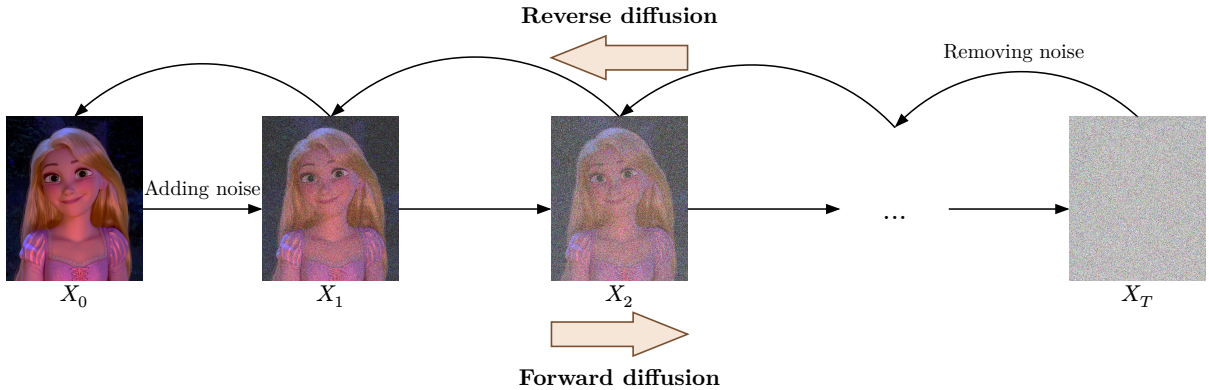


Figure 3: Complete diffusion process

3 Binary Diffusion Models

The binary diffusion models are a specific type of diffusion models that are designed to handle binary data, which is a common type of quantized data. For instance, in the context of computational imaging, we can deal with binary representations of images by a decomposition of the image into bit-planes or by binarization of the image using some neural network-based binarization method.

In binary diffusion models, thanks to the binary nature of the data, the management of the noise addition and removal process is simplified compared to the general case of continuous data. The forward diffusion process can be defined as a simple XOR operation between the binary data and a binary noise term, like the reverse diffusion process that is also defined as a simple XOR operation between the noisy binary data and a predicted binary noise term.

Thanks to the binary nature of the data, there is no need to tune a complex noise schedule as in the case of continuous data. This allows for a simpler preprocessing of the data and a more straightforward training of the model, which can lead to faster convergence and improved performance when dealing with binary data.

3.1 Training the model

The training of the binary diffusion model involves optimizing loss functions that encourage the model to learn how to effectively reverse the noise addition process and recover the original binary data from the noisy binary data.

The complete process can be represented as in the Figure 4, where the model learns to predict the original binary data $\hat{X}_{0_t}$ and the predicted noise $\hat{z}_t$ from the noisy binary data X_t at each time step t . In this way, the model can reverse the noise either by predicting the original binary data $\hat{X}_{0_t}$ (“target” mode), or by directly predicting the noise $\hat{z}_t$ and then applying the XOR operation with the noisy binary data X_t to remove the noise and recover the original data (“mask” mode).

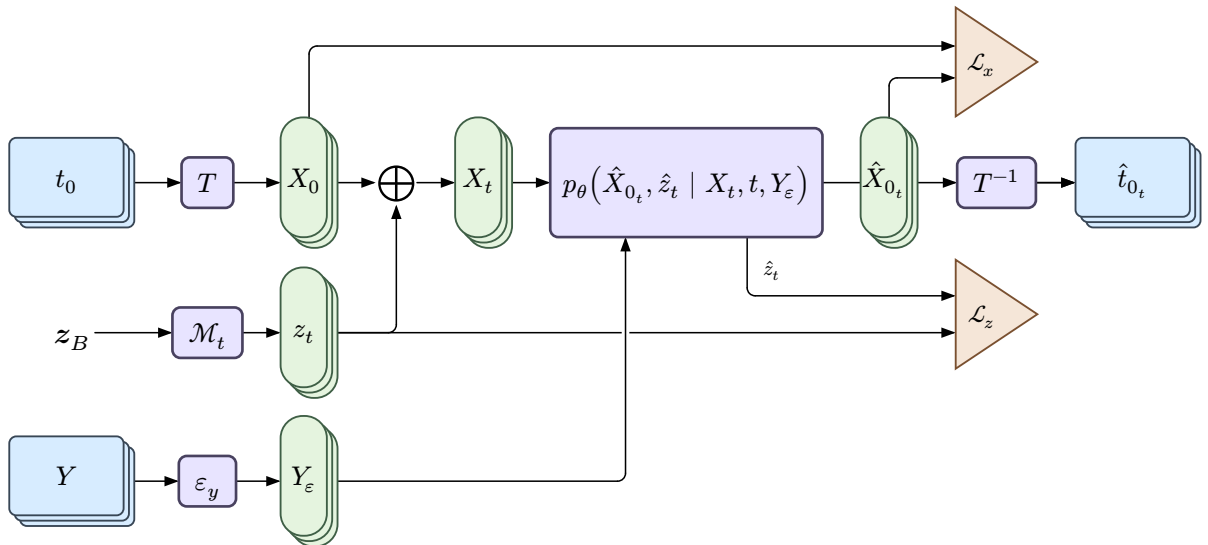


Figure 4: Binary diffusion process



As shown in Figure 4, the model p_θ takes also as input some additional information Y_ε that can be used to condition the generation process on some specific attributes of the data. This can be seen as the prompt of the user that guides the generation process towards specific types of samples.

3.1.1 Loss functions

Two loss functions are computed to train the model:

- $\mathcal{L}_z$: A loss function that encourages the model to accurately predict the noise $\hat{z}_t$ that was added to the data.
- $\mathcal{L}_x$: A loss function that encourages the model to accurately predict the original binary data $\hat{X}_{0_t}$ from the noisy binary data X_t .

In the paper “Tabular Data Generation using Binary Diffusion” by V. Kinakh and S. Voloshynovskiy [2], the authors propose the binary cross-entropy as the loss function for both $\mathcal{L}_z$ and $\mathcal{L}_x$, which is a common choice for binary data as it measures how well the predicted probabilities match the true binary labels.

$$\mathcal{L}_z = - \sum_{i=1}^N z_{t,i} \log(\hat{z}_{t,i}) + (1 - z_{t,i}) \log(1 - \hat{z}_{t,i}) \quad (9)$$

$$\mathcal{L}_x = - \sum_{i=1}^N X_{0,i} \log(\hat{X}_{0,i}) + (1 - X_{0,i}) \log(1 - \hat{X}_{0,i}) \quad (10)$$

In the Equation 9 and Equation 10:

- N is the number of binary features in the data.
- $z_{t,i}$ is the true binary noise for feature i at time step t .
- $\hat{z}_{t,i}$ is the predicted probability of the noise for feature i at time step t .
- $X_{0,i}$ is the true binary value of feature i in the original data.
- $\hat{X}_{0,i}$ is the predicted probability of the original binary value for feature i .

The overall loss function for training the model is then simply the mean of the two loss functions, as shown in Equation 11 where B denotes the batch size used during training and θ the parameters of the model to be optimized.

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{b=1}^B (\mathcal{L}_z^{(b)} + \mathcal{L}_x^{(b)}) \quad (11)$$

3.1.2 First transformation

The first transformation T can be defined as a simple binary encoding of the original data that must be invertible (T^{-1}) to ensure that the model can recover the original data from the noisy data.

For instance, in the context of computational imaging, we can decompose an image into bit-planes, where each bit-plane represents a binary image that captures a specific bit of the pixel values, as illustrated in Figure 5, with the k -th bit-plane representing the k -th bit of the pixel values.

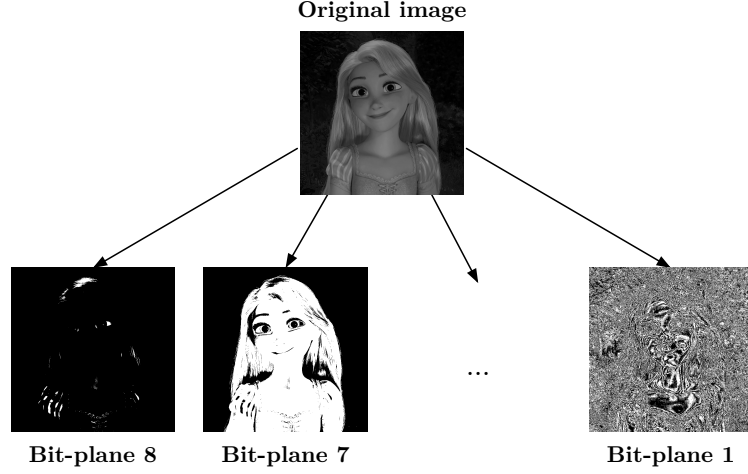


Figure 5: Example of a binary encoding of an image into bit-planes

So the first transformation T is important since it allows to work with different types of data such as images [3], tabular data [2] etc... by simply applying an appropriate binary encoding to the data.

3.2 Sampling from the model

The sampling process from the binary diffusion model involves starting from pure noise (the noisiest data) and iteratively applying the reverse diffusion process to generate a sample that resembles the original data.

It can be illustrated as in Figure 6, where the sampling process starts from pure noise X_T and iteratively applies the reverse diffusion process to generate a sample $\hat{X}_{0_t}$ that resembles the original data.

Note that the sampling process can also be guided by conditioning variables such as Y_ϵ to generate samples that have specific desired properties.

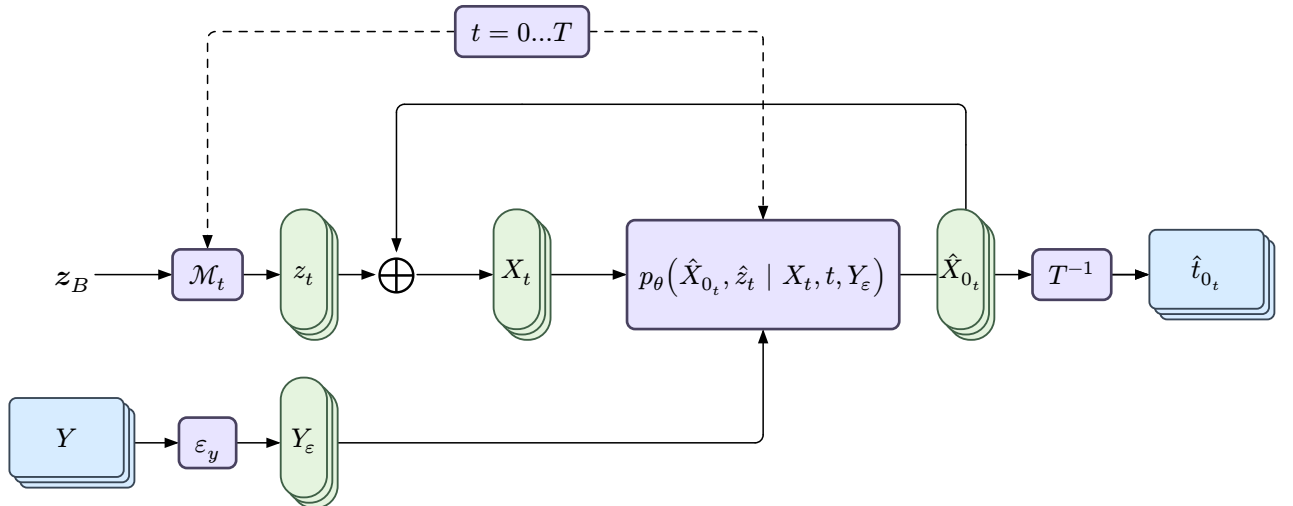


Figure 6: Sampling process

On the Figure 6, the noise depends on the time step t and is computed by a noise scheduler $\mathcal{M}_t$ that determines the amount of noise to be removed to granually denoise the data.



4 Binary Diffusion Models for Tabular Data Generation

Before working with images which are quite complex data and require more computational resources, we can start by working with tabular data which is simpler and more manageable, and then we can extend the approach to images.

In the paper “Tabular Data Generation using Binary Diffusion” by V. Kinakh and S. Voloshynovskiy [2], the authors propose a binary diffusion model for generating synthetic tabular data, which is a common type of quantized data that can be represented in a binary format.

The model is trained to learn the underlying distribution of the tabular data and can be used to generate new synthetic samples that resemble the original data (which can be useful for data augmentation, privacy preservation, etc...).

4.1 Link with image generation

The process of generating tabular data is similar to the process of generating images, the main difference being the first transformation T and the fact that the data are 1-dimensional instead of 2-dimensional as in the case of images.

4.2 First transformation for tabular data

The first transformation T for tabular data can be defined as a simple binary encoding of the original data, where the categorical features are one-hot encoded and the numerical features are binarized using a fixed number of bits to represent the values.

Then, all binary features are concatenated to form a single binary vector that represents the original tabular data, that can be reversed using the inverse transformation T^{-1} to recover the original tabular data from the binary representation.

4.3 Architecture of the model

As described in the paper [2], the sampling algorithm for generating synthetic data from the binary diffusion model can be represented as in the Algorithm 1.



Algorithm 1: Sampling Algorithm

```

1:  $X_t \leftarrow$  random binary tensor ▷ Initialize noisy sample
2:  $Y \leftarrow$  condition/label
3:  $Y_\varepsilon \leftarrow$  apply condition
4: threshold  $\leftarrow$  threshold value for binarization
5:  $p_\theta \leftarrow$  pre-trained denoiser network
6: for  $t \in \{T, \dots, 0\}$  do
7:    $\hat{X}_0, \hat{z}_t \leftarrow p_\theta(\hat{X}_0, \hat{z}_t \mid X_t, t, Y_\varepsilon)$  ▷ Predict clean sample and latent noise
8:    $\hat{X}_0 \leftarrow \sigma(\hat{X}_0) > \text{threshold}$  ▷ Apply sigmoid and binarize
9:    $\hat{z}_t \leftarrow \sigma(\hat{z}_t) > \text{threshold}$ 
10:   $z_t \leftarrow \text{get\_binary\_noise}(t-1)$  ▷ Generate random binary noise for time step t-1
11:   $X_t \leftarrow \hat{X}_0 \oplus z_t$  ▷ Update sample using XOR
12: end
13: return  $X_t$ 

```

4.3.1 Evaluation of the model

To evaluate the performance of the binary diffusion model for tabular data generation, the procedure described in the paper [4] is followed.

The evaluation procedure consists of the following steps:

1. Train the binary diffusion model on the original dataset.
2. Generate a synthetic dataset by sampling from the trained model.
3. Train some model on the synthetic dataset.
4. Evaluate the performances.

In the paper [2], the authors use a Random Forest classifier, a Linear/Logistic Regression classifier, and a Decision Tree classifier to evaluate the performance of the generated synthetic data on a classification and regression tasks, and they report the results in terms of prediction accuracy for classification tasks and mean squared error for regression tasks.

4.3.2 Fixed thresholding step

In the paper [2], the authors mention that the model doesn't perform well when increasing the number of time steps T in the reverse diffusion process.

This behavior is quite surprising since increasing the number of time steps T should allow the model to generate higher quality samples by gradually removing the noise from the data, which is a common behavior observed in diffusion models for continuous data.

However, note that in the Algorithm 1, the binarization step uses a fixed threshold value, which can be the root cause of the performance degradation.

For instance, if the model is in mode "mask", the predicted noise $\hat{z}_t$ can have 50% of ones meaning that the model will bring 50% of changes to the data at each step of the reverse diffusion process. This can degrade the quality of the generated samples since the model will be forced to make a large number of changes to the data at each step instead of gradually removing the noise.

Some tests have been done to try to understand the impact of the fixed thresholding step on the performance of the model



5 Results of Tabular Data Generation

First, I managed to reproduce the results of the paper [2] on the Adult dataset [5].

Then, I tried to improve the performance of the model by bringing some modifications:

- Removing the fixed thresholding
- Correctly applying the guidance during the sampling process
- Fixing the noise addition step in the sampling algorithm

5.1 Removing the fixed thresholding step

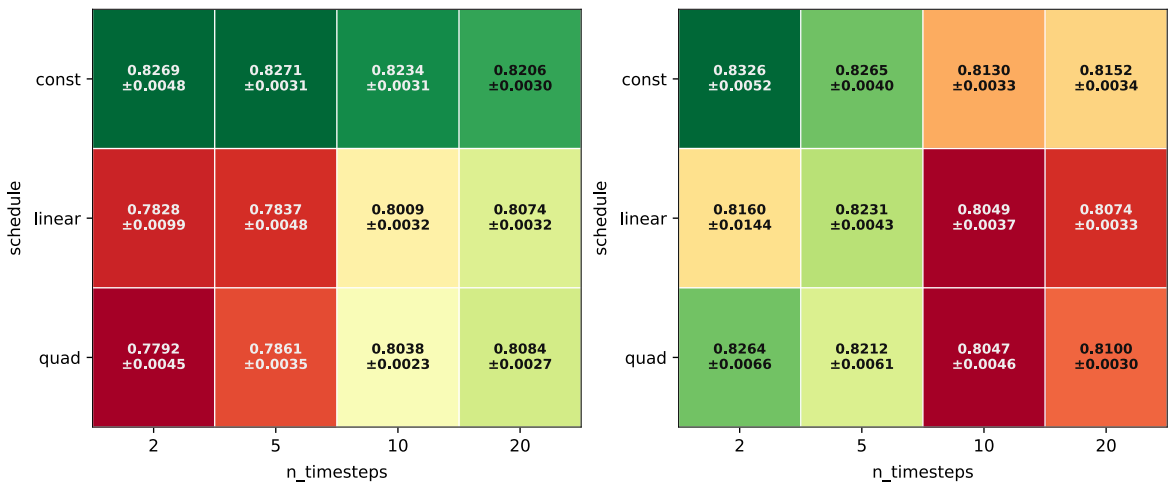
The first modification was to remove the fixed thresholding step in the sampling algorithm (mode “mask”).

Instead of applying a fixed threshold to binarize the predicted clean sample $\hat{X}_0$, I applied a dynamic threshold that is computed based on the quantity of noise to be removed at each step of the reverse diffusion process.

This dynamic thresholding approach allows the model to adapt the binarization process to the amount of noise that is being removed at each step, which can lead to a gradual removal of the noise and an improvement in the quality of the generated samples as we increase the number of time steps T in the reverse diffusion process.

The Figure 7 shows the results of the evaluation of the model with the dynamic thresholding approach (linear and quadratic schedules) compared to the fixed thresholding approach (constant schedule) for different values of T .

We can see that the dynamic thresholding approach doesn't improve at all the performance of the model compared to the fixed thresholding approach. And between the two dynamic thresholding approaches, the linear schedule performs worse than the quadratic schedule, which is expected since the model was trained with a quadratic noise schedule.



(a) use_t_next=False (b) use_t_next=True
Figure 7: Schedule comparison for strategy=mask (baseline: schedule=const)



Indeed, the model already predicts the right amount of noise to be removed at each step, as shown by the following results that compare the quantity of noise in the predicted mask with the quantity of noise that should be removed at each step of the reverse diffusion process.

Step 800: Ones in mask: 33.6364%, Ones required: 33.6031%
 Step 600: Ones in mask: 20.4545%, Ones required: 20.3957%
 Step 400: Ones in mask: 10.4545%, Ones required: 10.4685%
 Step 200: Ones in mask: 4.0909%, Ones required: 3.8215%
 Step 0: Ones in mask: 0.4332%, Ones required: 0.4545%

So this analysis shows that the fixed thresholding step is not the cause of the performance degradation since the model is already predicting the right amount of noise to be removed at each step, and the dynamic thresholding approach doesn't improve the performance of the model compared to the fixed thresholding approach.

5.2 Correctly applying the guidance during the sampling process

The second modification was to follow the approach described in the paper "Classifier-Free Diffusion Guidance" [6] to guide correctly the sampling process. In fact, the guidance was not correctly applied in the current implementation since it was not following the approach proposed in the paper, even though the goal was to implement the classifier-free guidance as described in the paper [6].

In the paper, the approach proposed is the following:

$$(1 + w) \cdot \varepsilon_c - w \cdot \varepsilon_u \quad (12)$$

Where ε_c is conditioned prediction and ε_u is unconditioned prediction, and w is a scalar value that controls the strength of the guidance.

The conditioned prediction can be seen as the conformity to the desired properties, while the unconditioned prediction can be seen as the diversity of the generated samples, and the guidance allows to find a good balance between these two aspects to generate high-quality samples that match the desired properties while still being diverse and representative of the general distribution of the data.

The current implementation was doing the following:

$$\varepsilon_u + g \cdot (\varepsilon_c - \varepsilon_u) \quad (13)$$

which is equivalent to the approach proposed in the paper [6] but with a different parameterization of the guidance strength: $g = 1 + w$:

$$\varepsilon_u + (1 + w) \cdot (\varepsilon_c - \varepsilon_u) = (1 + w) \cdot \varepsilon_c - w \cdot \varepsilon_u \quad (14)$$

This modification allows to correctly apply the guidance during the sampling process, but this is only a small fix of -1 in the implementation that doesn't have a significant impact on the performance of the model.

5.3 Fixing the noise addition step in the sampling algorithm

The third modification was a small fix in the implementation of the sampling algorithm.

As described in the Algorithm 1 at line 11, the sample process adds the noise of the step $t - 1$ to the predicted clean sample $\hat{X}_0$ to get the new sample X_{t-1} for the next step of the reverse diffusion process.

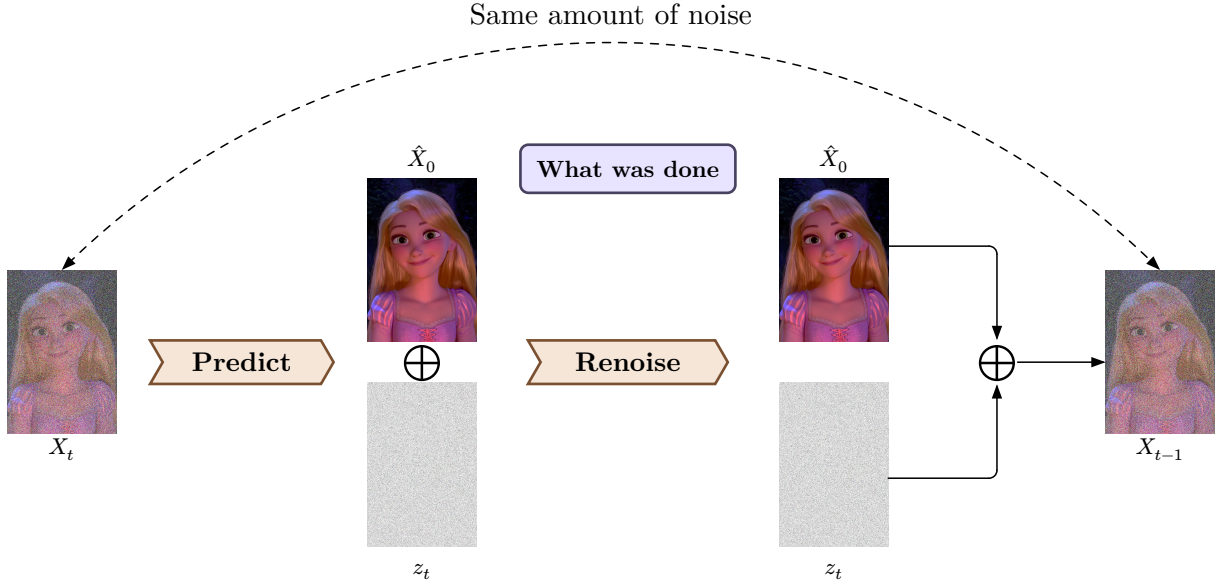


Figure 8: Current implementation of the noise addition step in the sampling algorithm

But in the existent implementation, the noise of the current step t was added instead of the noise of the next step $t - 1$ as shown in Figure 8.

The noise of the current step t is not the correct noise to be added since it corresponds to the noise that was removed at the current step, while the noise of the next step $t - 1$ corresponds to the noise that will be removed at the next step. The correct implementation should add the noise of the next step $t - 1$ to the predicted clean sample $\hat{X}_0$ to get the new sample X_{t-1} for the next step of the reverse diffusion process, as shown in Figure 9.

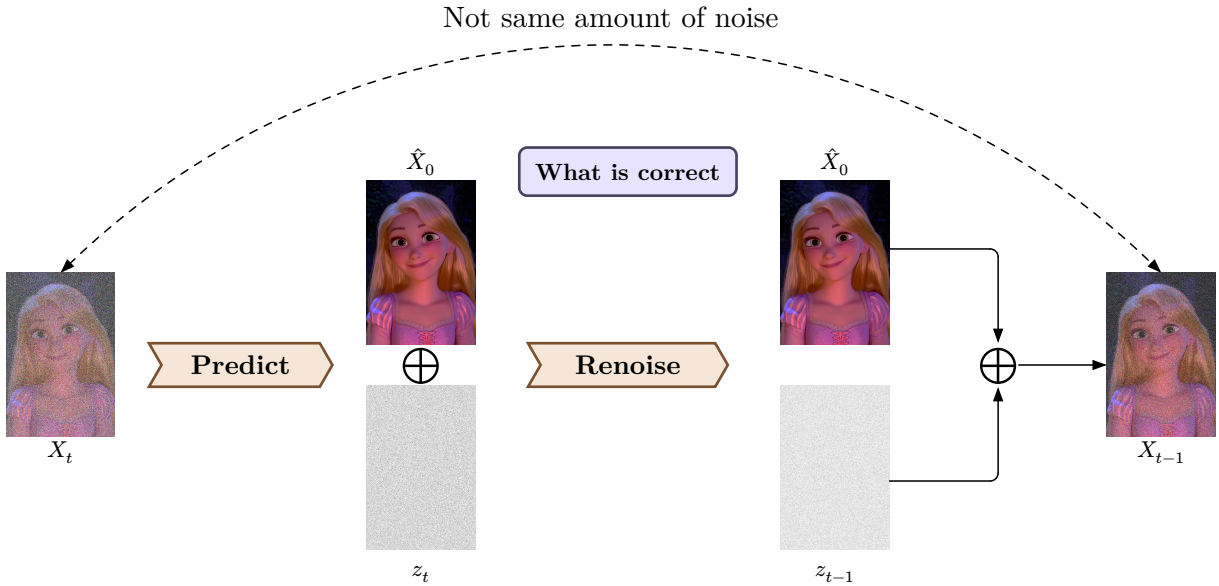


Figure 9: Correct implementation of the noise addition step in the sampling algorithm



By adding the correct noise at each step, the amount of noise is now correctly reduced at each step of the reverse diffusion process, and this fix is called `use_t_next` since it uses the noise of the next step $t - 1$ instead of the noise of the current step t .

However, as shown in Figure 10, this modification allows to generate better samples for small values of T but it doesn't allow to improve the performance of the model for larger values of T .

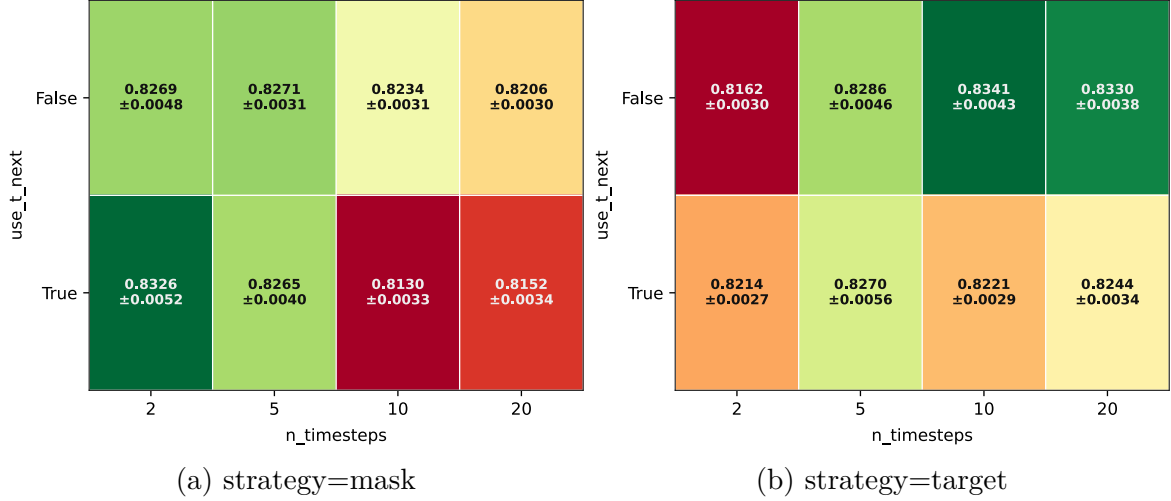


Figure 10: Fixed vs. non-fixed noise addition for different strategies
(baseline: `use_t_next=False`)

5.3.1 Increasing the noise in the sampling process

The last modification was to increase the noise in the sampling process by multiplying the predicted noise by a `renoise_factor` that is greater than 1, which allows to add more noise at each step of the reverse diffusion process.

This modification allows to better explore the space of possible samples, which results in an improvement of the performance of the model for larger values of T in the reverse diffusion process, as shown in Figure 11.

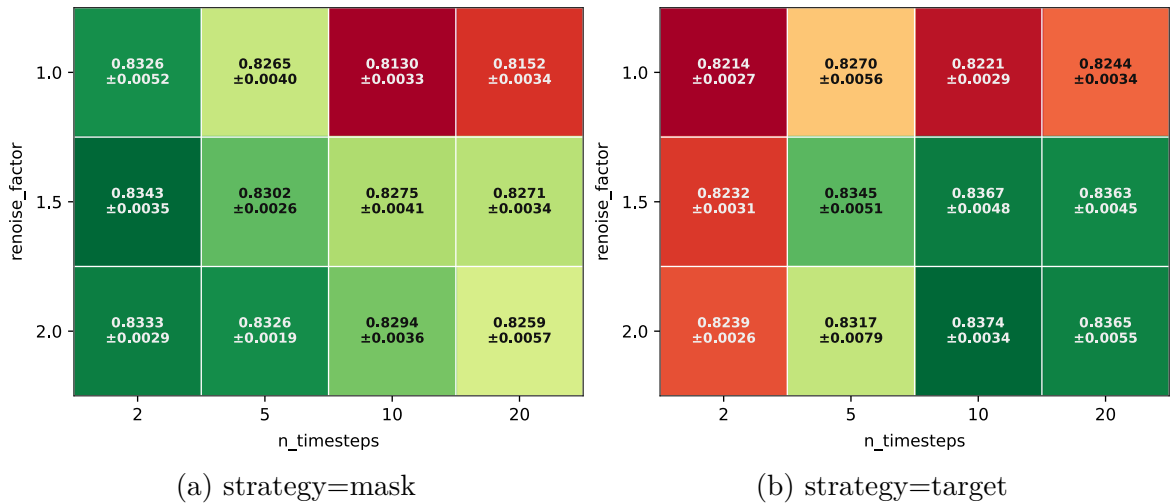


Figure 11: Increase the noise in the sampling process (`renoise_factor`) for `use_t_next=True`
(baseline: `renoise_factor=1.0`)



The Table 1 shows the best configurations for each strategy compared to the original implementation of the model (baseline) for different values of T in the reverse diffusion process. We can see that the modifications allow to improve the performance of the model of up to 1% for different configurations, which shows that the modifications have a positive impact on the performance of the model, but there is still room for improvement to further enhance the performance of the model for larger values of T in the reverse diffusion process.

10 configurations — Dataset: adult | Metric: accuracy | Sorted best → worst | Baseline (blue row): 0.8268

	rank	dataset	strategy	use_t_next	renoise_factor	n_timesteps	schedule	DT	LR	RF	mean_all_models	std_all_models	delta_vs_baseline
0	BASE	adult	target	True	1.000000	5	const	0.8206	0.8377	0.8221	0.8268	0.0094	0.0000
1	1	adult	target	True	2.000000	10	const	0.8333	0.8464	0.8326	0.8374	0.0078	0.0106
2	2	adult	target	True	1.500000	10	const	0.8337	0.8475	0.8290	0.8367	0.0096	0.0099
3	3	adult	target	True	2.000000	20	const	0.8307	0.8463	0.8325	0.8365	0.0085	0.0097
4	4	adult	target	True	1.500000	20	const	0.8349	0.8446	0.8295	0.8363	0.0077	0.0095
5	5	adult	target	True	1.500000	5	const	0.8309	0.8447	0.8279	0.8345	0.0090	0.0076
6	6	adult	mask	True	1.500000	2	const	0.8283	0.8403	0.8345	0.8343	0.0060	0.0075
7	7	adult	target	False	1.000000	10	const	0.8332	0.8409	0.8282	0.8341	0.0064	0.0073
8	8	adult	mask	True	2.000000	2	const	0.8350	0.8353	0.8298	0.8333	0.0031	0.0065
9	9	adult	target	False	1.000000	20	const	0.8318	0.8397	0.8275	0.8330	0.0062	0.0062
10	10	adult	mask	True	2.000000	5	const	0.8335	0.8381	0.8263	0.8326	0.0060	0.0058

Table 1: Best configurations vs. original implementation (baseline)

Here we were dealing with tabular data which is simpler than images, but the same modifications can be applied to the case of image generation to try to improve the performance of the model for larger values of T in the reverse diffusion process.

These modifications can be much more impactful in the case of images since the data are more complex and the noise addition and removal process is more challenging compared to the case of tabular data.



6 Results of Image Generation

After working on the tabular data generation, I started to work on the image generation using binary diffusion models, and I tried to apply the same modifications that were applied to the case of tabular data generation to see if they can also improve the performance of the model.

The results of the evaluation of the model were obtained after 5 days of computation on a RTX 4080 (16 Go VRAM), which shows that working with images is much more computationally expensive than working with tabular data, and it requires more time and resources to test and evaluate.

6.1 Evaluation of the model

To evaluate the performance of the binary diffusion model for image generation, I used the CIFAR-10 dataset [7] that was used for the training of the model.

The evaluation procedure consists of the following steps:

1. Train the binary diffusion model on the CIFAR-10 dataset.
2. Generate a synthetic dataset by sampling from the trained model.
3. Evaluate the quality of the generated images using metrics such as Inception Score (IS) Fréchet Inception Distance (FID) and Kernel Inception Distance (KID) that are commonly used for evaluating the performance of generative models for images.

The IS measures the quality of the generated images by evaluating how well they can be classified by a pre-trained Inception model, while the FID measures the distance between the distribution of the generated images and the distribution of the real images in the feature space of a pre-trained Inception model.

For the evaluation, the IS, FID, FID-clip and KID metrics were used, where FID-clip is a variant of the FID metric that uses a pre-trained CLIP model instead of an Inception model and KID is a variant of the FID metric that uses a kernel-based approach to measure the distance between the distributions of the generated and real images.

6.1.1 How to interpret the results of the evaluation

- IS: Higher is better
- FID: Lower is better
- FID-clip: Lower is better
- KID: Lower is better

6.2 Fixing the thresholding step in the sampling algorithm

The same test of fixing the thresholding step to a certain amount of noise to be removed at each step of the reverse diffusion process was applied to the case of image generation, but it doesn't allow to improve the performance of the model, as shown in Figure 12.

This is consistent with the results obtained for the case of tabular data generation since in both cases the model already predicts the right amount of noise to be removed at each step.



The only exception is the case of the FID-clip metric where the linear and quadratic schedules perform better than the baseline.

schedule	False		True	
	FID	FID-C	FID	FID-C
const	151.3677	25.4344	63.5944	15.0841
	IS	KID	IS	KID
linear	219.1403	33.9367	67.3314	15.8181
	IS	KID	IS	KID
quad	219.1860	33.9804	67.5868	15.8412
	IS	KID	IS	KID

use_t_next

(a) n_timesteps=10

schedule	False		True	
	FID	FID-C	FID	FID-C
const	22.1167	6.4232	20.0589	6.8247
	IS	KID	IS	KID
linear	24.8353	7.8071	21.1195	6.5216
	IS	KID	IS	KID
quad	24.8312	7.7766	21.0930	6.5343
	IS	KID	IS	KID

use_t_next

(b) n_timesteps=100

Figure 12: Schedule comparison for strategy=mask
(baseline: schedule=const)

6.3 Fixing the noise addition step in the sampling algorithm

The same test of fixing the noise addition step to add the correct noise at each step of the reverse diffusion process was applied to the case of image generation, leading to a big improvement of the performance of the model for smaller values of T in the reverse diffusion process, and only a small improvement for larger values of T , as shown in Figure 13.

For larger values of T , the FID-clip metric is the only metric that shows a decrease in the performance of the model when fixing the noise addition step, which is quite surprising since this modification should allow to improve the performance of the model by ensuring that the correct amount of noise is removed at each step of the reverse diffusion process.

use_t_next	False		True	
	FID	FID-C	FID	FID-C
False	152.0496	25.6824	143.9530	24.7038
	IS	KID	IS	KID
True	65.0158	15.2561	63.1987	14.8850
	IS	KID	IS	KID

strategy

(a) n_timesteps=10

use_t_next	False		True	
	FID	FID-C	FID	FID-C
False	21.2099	6.3432	20.8752	6.3321
	IS	KID	IS	KID
True	20.1612	6.8149	19.7619	6.7677
	IS	KID	IS	KID

strategy

(b) n_timesteps=100

Figure 13: Fixed vs. non-fixed noise addition (use_t_next) for different metrics and strategies
(baseline: use_t_next=False)

So fixing the noise addition step allows to improve globally the performance of the model, making it more effective at reversing the noise addition process and generating higher quality samples, but it doesn't allow to improve significantly the performance of the model for larger values of T in the reverse diffusion process, which indicates that there are still some issues that need to be addressed to further enhance the performance of the model for larger values of T .

Compared to the case of tabular data generation, fixing the noise addition step has a much bigger impact on the performance of the model for image generation, which can be explained by the fact that images are more complex data than tabular data, and the noise addition and removal process is more challenging for images compared to tabular data.

6.4 Increasing the noise in the sampling process

The same test of increasing the noise in the sampling process by multiplying the predicted noise by a renoise factor that is greater than 1 was applied to the case of image generation, leading to a significant decrease in the performance of the model for larger values of T in the reverse diffusion process, as shown in Figure 14.

This result is expected since increasing the noise means degrading the quality of the generated samples since the model will not be able to remove the noise effectively.

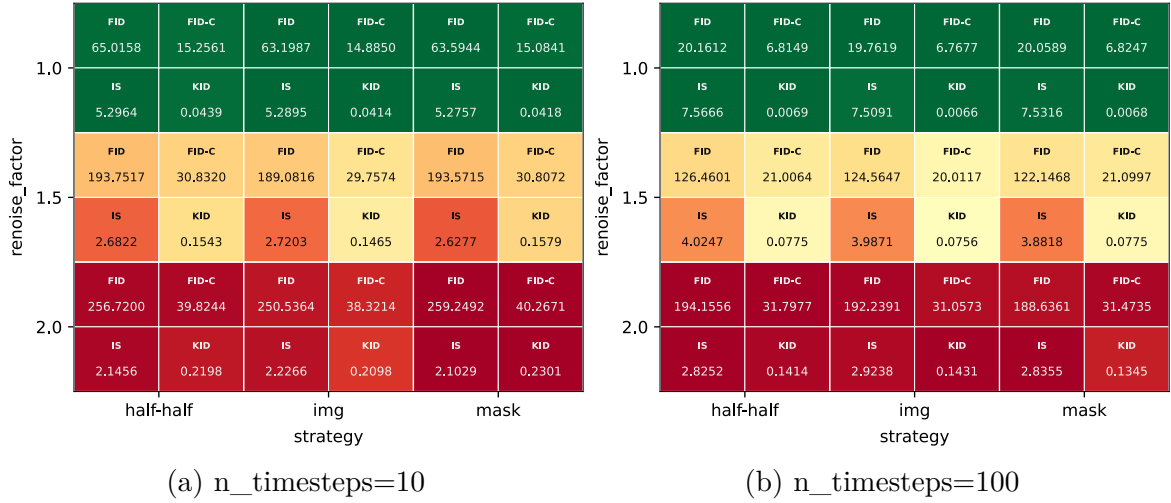


Figure 14: Increase the noise in the sampling process ($renoise_factor$) for $use_t_next=True$ (baseline: $renoise_factor=1.0$)

6.5 Generated images

The Figure 15 shows some examples of generated images for different values of T . We can distinguish horses in both images, but the image generated with $T = 100$ shows more details and better quality.



(a) $T = 10$



(b) $T = 100$

Figure 15: Generated images for different values of T in the reverse diffusion process



7 Conclusion

In this report, we explored the concept of binary diffusion models and their application to both tabular data generation and image generation.

We started by understanding the complete diffusion process, which consists of a forward diffusion process that adds noise to the data and a reverse diffusion process that learns to remove the noise and recover the original data.

Then, we focused on binary diffusion models, which are designed to handle binary data, and we discussed the training of the model, including the loss functions used to optimize the model and the first transformation that allows to work with different types of data.

Next, we evaluated the performance of the binary diffusion model for tabular data generation and image generation, and we applied some modifications to try to improve the performance of the model for larger values of T in the reverse diffusion process. The modifications included removing the fixed thresholding step, correctly applying the guidance during the sampling process, fixing the noise addition step, and increasing the noise in the sampling process.

Overall, the fixing of the noise addition step was the most impactful modification that allowed to improve the performance of the model for both tabular data generation and image generation, while the other modifications had a smaller impact on the performance of the model.

But there is still room for improvement to further enhance the performance of the model for larger values of T in the reverse diffusion process, and future work can focus on addressing the remaining issues and exploring other modifications that can further improve the performance of the model.

7.1 Acknowledgements

I would like to thank V. Kinakh, the author of the papers “Tabular Data Generation using Binary Diffusion” [2] and “Binary Diffusion Probabilistic Model” [3], for his guidance and support throughout this project, and for providing investigations directions that helped to understand the issues with the model and how to address them.

7.2 Usage of AI

- Claude Sonnet 4.6 (Free): code generation, code debugging, code review, graph library generation.
- ChatGPT: explanations of concepts, code debugging.
- Github Copilot autocompletion: code writing and report writing.



Bibliography

- [1] L. Weng, “What are diffusion models?,” *lilianweng.github.io*, July 2021, [Online]. Available: <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>
- [2] V. Kinakh and S. Voloshynovskiy, “Tabular Data Generation using Binary Diffusion.” [Online]. Available: <https://arxiv.org/abs/2409.13882>
- [3] V. Kinakh and S. Voloshynovskiy, “Binary Diffusion Probabilistic Model.” [Online]. Available: <https://arxiv.org/abs/2501.13915>
- [4] V. Borisov, K. Seßler, T. Leemann, M. Pawelczyk, and G. Kasneci, “Language Models are Realistic Tabular Data Generators.” [Online]. Available: <https://arxiv.org/abs/2210.06280>
- [5] B. Becker and R. Kohavi, “Adult.” 1996.
- [6] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance.” [Online]. Available: <https://arxiv.org/abs/2207.12598>
- [7] A. Krizhevsky, “Learning multiple layers of features from tiny images,” technical report, 2009.